

Aula 11

*DataPrev (Perfil 3: Desenvolvimento de Software) Engenharia de Software - 2026
(Pós-Edital)*

Autor:

**Diego Carvalho, Equipe
Informática 2 (Diego Carvalho),
Equipe Informática 1 (Diego
Carvalho)**

21 de Julho de 2026

Índice

1) Padrões de Projeto - Teoria	3
2) Padrões de Projeto - Questões Comentadas - FGV	18
3) Padrões de Projeto - Lista de Questões - FGV	32

APRESENTAÇÃO

Fala, pessoal! Tudo bem?

A disciplina de Padrões de Projeto é frequentemente considerada maçante e bastante "decoreba", exigindo muita memorização dos alunos. Para quem tem bastante tempo, como em um estudo pré-edital, a orientação é realizar uma leitura completa do material. O aluno deve focar inicialmente nos Conceitos Básicos, que apresentam incidência altíssima nas provas, e avançar para a compreensão detalhada de todos os padrões. Embora o conteúdo seja denso, ter tempo permite dominar os tópicos de incidência média e alta, garantindo pontos valiosos.

Para o aluno em reta final pós-edital com o tempo esgotado, a estratégia foca puramente no melhor custo-benefício. Como a aula é muito teórica, memorize as frases e mnemônicos iniciais, pois garantem o acerto de várias questões rapidamente. Além disso, descarte quase todos os Padrões Comportamentais, que é a mais extensa e toma muito tempo (foque-se naqueles com incidência maior). Aliás, analise com cuidado as incidências de cada padrão em prova.

Prof. Diego Carvalho

📷 **INSTAGRAM DO PROFESSOR- [WWW.INSTAGRAM.COM/PROFESSORDIEGOCARVALHO](https://www.instagram.com/professordiegocarvalho)**

AVENTURAS EM DESIGN PATTERNS: O CASO DO SINGLETON



PADRÕES DE PROJETO

Conceitos Básicos

INCIDÊNCIA EM PROVA: ALTÍSSIMA

PADRÕES DE PROJETO GOF

Os Padrões de Projeto GoF (Gang of Four) são um conjunto de 23 soluções reutilizáveis para problemas recorrentes no desenvolvimento de software orientado a objetos, apresentados no livro *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), de Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides. Esses padrões não representam implementações prontas, mas sim modelos de solução que orientam a organização das classes, objetos e suas interações, promovendo maior reutilização, flexibilidade, baixo acoplamento e facilidade de manutenção dos sistemas.

Antes de estudar os principais padrões de projeto, é importante compreender o conceito de padrão. **Um padrão corresponde a uma solução recorrente para um problema que também ocorre de forma recorrente em determinado contexto.** O que torna essa ideia tão útil? Em vez de criar uma solução completamente nova sempre que o mesmo problema surgir, torna-se possível aproveitar um modelo já documentado, analisado e validado em diferentes situações. Christopher Alexander sintetiza essa proposta ao afirmar que cada padrão descreve um problema recorrente e o núcleo de uma solução que pode ser reutilizada inúmeras vezes, sem que sua implementação precise ser exatamente igual em todos os casos.

Com base nessa ideia, surgiram os padrões de projeto, conhecidos como Design Patterns. Segundo a definição apresentada por Gamma, Helm, Johnson e Vlissides, autores conhecidos como Gang of Four (GoF), **os padrões de projeto consistem em descrições de objetos que se comunicam e de classes personalizadas para resolver um problema genérico de projeto em um contexto específico.** Isso significa que um padrão corresponde a um trecho de código pronto para reutilização? Não. **Um padrão descreve uma solução abstrata que pode ser adaptada às necessidades de cada sistema** e de cada domínio de aplicação.

A utilidade dessa abordagem torna-se evidente quando se observa o desenvolvimento de sistemas para diferentes áreas de negócio. Embora as aplicações possuam finalidades distintas, muitos problemas de projeto se repetem ao longo do tempo. Em vez de desenvolver uma solução inteiramente nova para cada situação, os desenvolvedores podem recorrer a modelos já conhecidos e adaptá-los ao contexto específico do sistema. **Essa reutilização reduz o esforço de projeto, favorece a adoção de soluções consolidadas e facilita a comunicação entre profissionais** que compartilham uma mesma terminologia.

A origem desse conceito não está na Engenharia de Software. Como essa ideia chegou ao desenvolvimento de programas? A inspiração veio da Arquitetura. **Na década de 1970, Christopher Alexander dedicou-se ao estudo de problemas recorrentes no planejamento de cidades**, edifícios e outros espaços construídos. **Em 1977, juntamente com Sara Ishikawa e Murray Silverstein, publicou a obra A Pattern Language**, na qual apresentou um conjunto de padrões destinados a orientar a elaboração de projetos em diferentes escalas. Cada padrão descrevia um problema recorrente e uma solução que poderia ser adaptada às características específicas de cada projeto.

A proposta chamou a atenção da comunidade de Engenharia de Software, que enfrentava desafios semelhantes no desenvolvimento de sistemas orientados a objetos. **Em 1994, Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides publicaram o livro Design Patterns: Elements of Reusable Object-Oriented Software.** Qual foi a principal contribuição dessa obra? Em vez de apresentar bibliotecas ou componentes prontos, **o livro documentou vinte e três padrões de projeto que**

descrevem soluções recorrentes para problemas comuns de projeto em software orientado a objetos. Essa publicação tornou-se uma das principais referências sobre o tema.

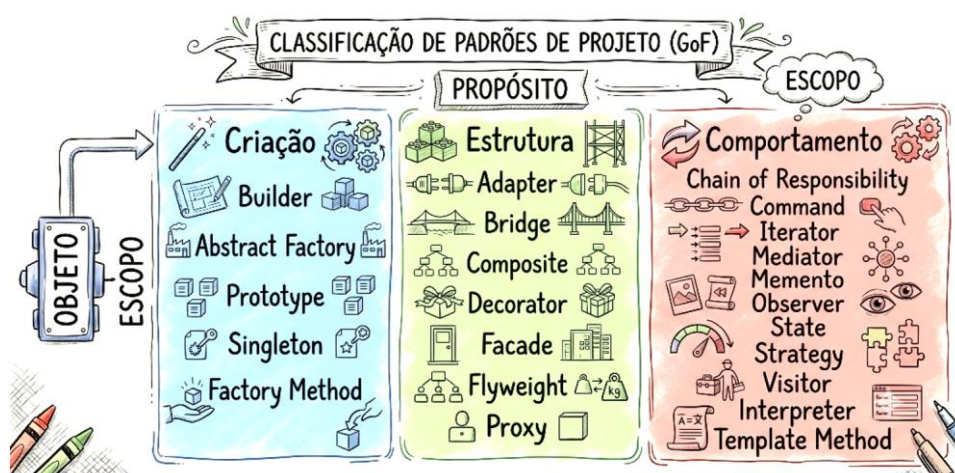
Os padrões de projeto oferecem diversas vantagens durante o desenvolvimento de software. Entre elas estão a reutilização de soluções já consolidadas, a adoção de uma terminologia comum entre os desenvolvedores, a melhoria da documentação dos projetos e o aproveitamento da experiência acumulada pela comunidade de Engenharia de Software. Além disso, **muitos padrões favorecem a construção de sistemas com maior coesão, menor acoplamento e melhor organização estrutural**, contribuindo para facilitar a manutenção e a evolução do código.

A utilização de padrões de projeto, entretanto, não elimina a necessidade de análise crítica. Seria adequado aplicar um padrão sempre que ele estiver disponível? Nem sempre. **A adoção inadequada de um padrão pode aumentar a complexidade da solução sem produzir benefícios proporcionais.** Em algumas situações, recursos oferecidos pelas linguagens modernas de programação, como funções de primeira classe, expressões lambda, delegação, mecanismos de composição e outros recursos da linguagem, tornam desnecessária a aplicação de determinados padrões. Por esse motivo, alguns autores defendem que certos Design Patterns surgiram para suprir limitações existentes em linguagens mais antigas, embora essa interpretação não seja unânime na literatura.

Cada padrão de projeto é descrito segundo uma estrutura padronizada composta por quatro elementos fundamentais. O que cada um deles representa? O nome identifica o padrão e estabelece uma terminologia comum entre os desenvolvedores. O problema descreve a situação em que sua utilização é recomendada. A solução apresenta a estrutura geral capaz de resolver o problema, sem impor uma implementação específica. Por fim, **as consequências descrevem os impactos decorrentes da adoção do padrão**, incluindo suas vantagens, limitações e os efeitos sobre a arquitetura do sistema.

Antes de estudar cada padrão individualmente, é importante compreender o alcance do catálogo elaborado pelo *Gang of Four*. Os vinte e três padrões podem ser aplicados em qualquer tipo de projeto de software? Não. **Esse catálogo foi desenvolvido especificamente para sistemas orientados a objetos** e utiliza conceitos característicos desse paradigma, como classes, objetos, herança, composição e polimorfismo. Isso não significa, entretanto, que os padrões de projeto existam apenas na orientação a objetos, pois há diversos outros padrões aplicáveis a diferentes paradigmas e estilos de desenvolvimento.

Os padrões do *Gang of Four* são tradicionalmente classificados em três grandes categorias, de acordo com a finalidade que desempenham durante o desenvolvimento de software. Como essa classificação é organizada? **Os padrões dividem-se em criacionais, estruturais e comportamentais.** Essa organização facilita o estudo do catálogo, pois reúne padrões que procuram resolver problemas semelhantes dentro de uma mesma categoria.



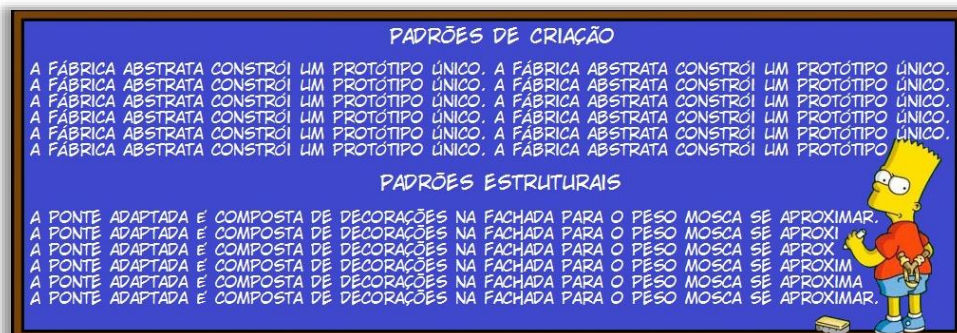
Os padrões criacionais concentram-se na forma como os objetos são criados. Em vez de permitir que cada parte do sistema realize diretamente a instanciação das classes, esses padrões procuram encapsular e flexibilizar o processo de criação, reduzindo o acoplamento entre os componentes e tornando o código mais adaptável a mudanças. Dessa forma, o mecanismo de criação dos objetos pode evoluir sem exigir alterações significativas nas partes que os utilizam.

Os padrões estruturais voltam-se para a organização das classes e dos objetos. O que caracteriza esse grupo de padrões? Seu objetivo é estabelecer formas eficientes de compor estruturas maiores a partir de componentes menores, favorecendo a reutilização, a flexibilidade e a redução do acoplamento entre os elementos do sistema. Assim, torna-se possível construir arquiteturas mais organizadas e mais fáceis de manter ao longo do ciclo de vida do software.

Os padrões comportamentais concentram-se na interação entre os objetos e na distribuição de responsabilidades durante a execução do sistema. Eles descrevem mecanismos que organizam a comunicação entre os componentes, definem como determinadas tarefas devem ser distribuídas e estabelecem formas padronizadas de colaboração entre objetos. Por esse motivo, compreender a classificação dos vinte e três padrões é um passo importante para identificar qual categoria oferece a solução mais adequada para cada problema de projeto.



Quando eu estudei esse assunto na minha vida de concurseiro, utilizei de um mnemônico maneiríssimo criado pelo Prof. Rogério Araújo para memorizar:



Explicação: A fábrica (*Factory Method*) abstrata (*Abstract Factory*) constrói (*Builder*) um protótipo (*Prototype*) único (*Singleton*). A ponte (*Bridge*) adaptada (*Adapter*) é composta (*Composite*) de decorações (*Decorator*) na fachada (*Facade*) para o peso-mosca (*Flyweight*) se aproximar (*Proxy*). E não tem frase para o último? Não, porque não é necessária! **Se não é um padrão criacional ou estrutural, é um padrão comportamental.**

Padrões Criacionais

PADRÕES CRIACIONAIS

Os Padrões Criacionais são uma categoria dos padrões GoF voltada para abstrair e controlar o processo de criação de objetos. Seu principal objetivo é desacoplar o código cliente da forma como os objetos são instanciados, tornando o sistema mais flexível e facilitando a substituição ou extensão das classes concretas. Fazem parte dessa categoria os padrões Abstract Factory, Builder, Factory Method, Prototype e Singleton.

Abstract Factory

INCIDÊNCIA EM PROVA: MÉDIA

Descrição Resumida: esse padrão fornece uma interface para criar famílias de objetos relacionados ou dependentes sem especificar suas classes concretas.

O padrão de projeto Abstract Factory **fornece uma interface para criar famílias de objetos relacionados ou dependentes sem que o código cliente precise conhecer suas classes concretas.** Essa abordagem desacopla o processo de criação dos objetos de sua utilização, pois o cliente trabalha apenas com interfaces. Além disso, o padrão garante que todos os objetos produzidos pertençam à mesma família e sejam compatíveis entre si.

Uma dúvida bastante comum surge nesse ponto: o que significa uma família de objetos? **Uma família corresponde a um conjunto de componentes projetados para funcionar em conjunto.** Imagine uma aplicação que possa ser executada tanto em computadores quanto em smartphones. Cada ambiente possui botões, menus e barras de navegação próprios, mas todos esses componentes pertencem à mesma interface gráfica. Em vez de criar cada elemento individualmente, uma fábrica concreta produz toda a família correspondente ao ambiente selecionado.

Outro aspecto importante envolve a escolha da fábrica concreta. Como isso acontece? Normalmente, o código cliente identifica algum parâmetro de configuração, uma preferência do usuário ou o ambiente de execução e, a partir dessa informação, instancia a fábrica adequada. Depois dessa escolha, todas as solicitações de criação de objetos passam pela interface da fábrica abstrata, enquanto as implementações concretas permanecem ocultas do restante da aplicação.

Vale também comparar esse padrão com outro bastante conhecido. Seriam o Factory Method e o Abstract Factory a mesma solução? Apesar de ambos encapsularem a criação de objetos, eles possuem objetivos diferentes. **O Factory Method normalmente cria um único tipo de produto e delega essa decisão às subclasses; o Abstract Factory, por sua vez, cria famílias completas de objetos relacionados,** garantindo que todos sejam compatíveis e possam ser utilizados em conjunto.

```
class FabricaWindows:
    def criar_botao(self):
        return "Botão do Windows"

    def criar_menu(self):
```



```
return "Menu do Windows"

class FabricaMac:
    def criar_botao(self):
        return "Botão do Mac"

    def criar_menu(self):
        return "Menu do Mac"

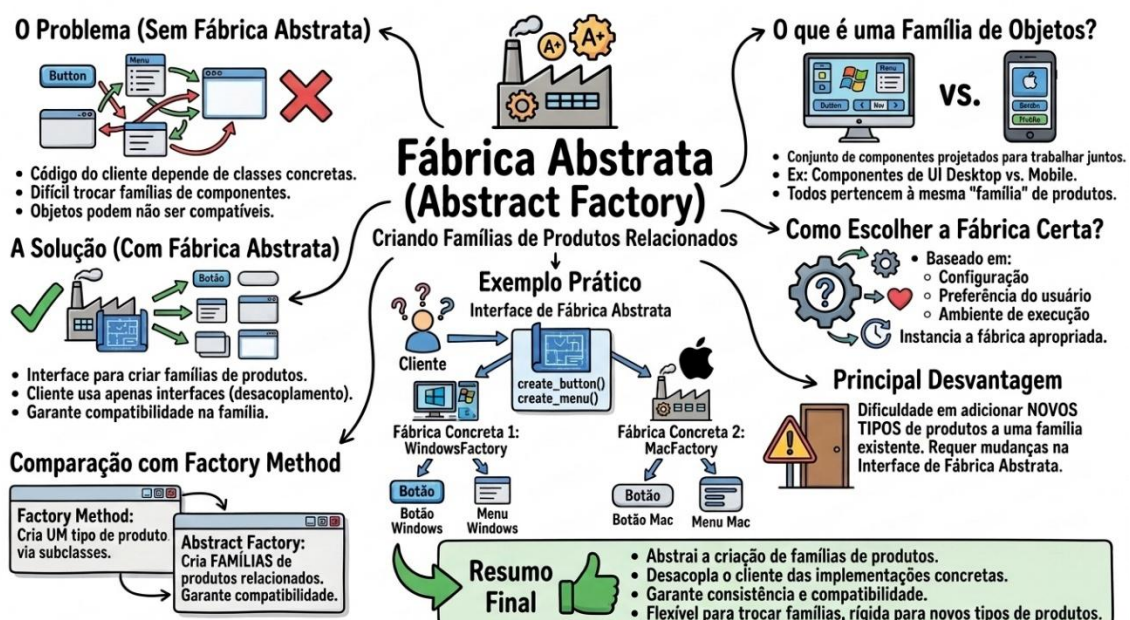
def criar_interface(fabrica):
    print(fabrica.criar_botao())
    print(fabrica.criar_menu())

criar_interface(FabricaWindows())
```

Nesse exemplo, FabricaWindows e FabricaMac representam fábricas diferentes. Cada uma cria uma família de objetos relacionados: um botão e um menu do mesmo sistema operacional. A função criar_interface() não precisa saber se está trabalhando com Windows ou Mac. Ela apenas recebe uma fábrica e solicita os objetos por meio dos métodos criar_botao() e criar_menu(). Para trocar toda a família de componentes, basta alterar a fábrica recebida: criar_interface(FabricaMac()).

A ideia central do Abstract Factory é justamente essa: fornecer uma forma de criar conjuntos de objetos relacionados sem fazer o código cliente depender diretamente das classes concretas desses objetos. **É importante destacar, entretanto, a principal desvantagem do Abstract Factory:** estender o sistema para adicionar novas "categorias" (novos tipos) de produtos em uma família já existente é uma tarefa trabalhosa.

A interface central (a fábrica abstrata) condensa todos os métodos de criação (como criar_botao() e criar_menu()). Se o sistema precisar passar a produzir um novo componente genérico (como criar_janela()), essa mudança **exigirá a alteração forçada na interface da Fábrica Abstrata e, consequentemente, a reescrita de todas as subclasses de fábricas concretas** vinculadas a ela, o que fere o Princípio Aberto/Fechado (OCP).



Factory Method

INCIDÊNCIA EM PROVA: ALTA

Descrição Resumida: esse padrão define uma interface para criar um objeto, mas deixa as subclasses decidirem qual classe instanciar.

O padrão de projeto Factory Method é utilizado quando uma classe não consegue determinar antecipadamente qual objeto concreto deverá instanciar. Em quais situações isso acontece? Geralmente quando diferentes objetos compartilham uma mesma interface ou classe base, mas a escolha da implementação concreta depende de informações conhecidas apenas durante a execução da aplicação. Assim, a lógica de criação permanece flexível e desacoplada do restante do sistema.

Considere uma concessionária que comercializa diversos modelos de automóveis. Em vez de criar uma classe independente para manipular cada modelo, define-se uma classe abstrata que representa um carro de forma genérica e, a partir dela, são criadas subclasses para modelos específicos, como Corsa, Celta, Cruze e Camaro. Por que adotar essa estrutura? Porque todos os modelos compartilham características comuns, mas cada um possui sua própria implementação.

A principal dificuldade surge quando o sistema precisa criar um objeto. Como decidir qual modelo deve ser instanciado? **O Factory Method resolve esse problema ao delegar essa responsabilidade às subclasses criadoras**, que implementam o método responsável pela criação do produto. Dessa forma, a escolha do objeto concreto pode ocorrer durante a execução da aplicação, conforme as regras de negócio, configurações ou informações disponíveis naquele momento.

A estrutura clássica desse padrão é formada pelos participantes Creator, ConcreteCreator, Product e ConcreteProduct. O que cada um representa? **O Product define a interface comum dos objetos criados; o ConcreteProduct implementa essa interface; o Creator declara o método fábrica**; por sua vez, o ConcreteCreator implementa esse método e determina qual produto concreto será instanciado em cada situação.

```
class Cachorro:
    def emitir_som(self):
        return "Au au!"

class Gato:
    def emitir_som(self):
        return "Miau!"

class FabricaAnimal:
    def criar_animal(self, tipo):
        if tipo == "cachorro":
            return Cachorro()
        elif tipo == "gato":
            return Gato()

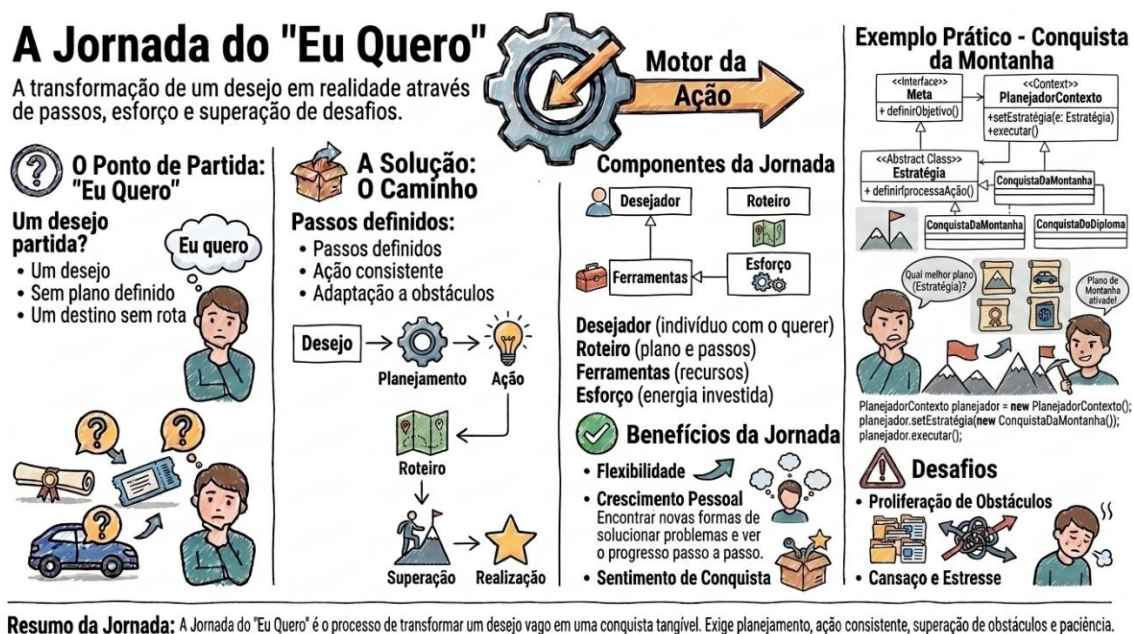
fabrica = FabricaAnimal()

animal = fabrica.criar_animal("gato")
print(animal.emitir_som())
```

No exemplo, as classes Cachorro e Gato representam os produtos que podem ser criados. A classe FabricaAnimal implementa o Factory Method, isto é, um método (criar_animal()) responsável por decidir qual objeto instanciar com base na solicitação recebida. A principal ideia do Factory Method é encapsular a criação de objetos em um método, evitando que o código cliente utilize diretamente comandos como Cachorro() ou Gato(). Em vez disso, o cliente solicita um objeto à fábrica, que decide qual classe concreta deve ser instanciada.

Esse padrão é útil quando o sistema pode precisar criar diferentes tipos de objetos sem que o código cliente conheça suas classes concretas. Assim, caso seja necessário adicionar um novo tipo de animal (como Pássaro), basta modificar ou estender a fábrica, mantendo o restante da aplicação desacoplado do processo de criação dos objetos.

A grande desvantagem de adotar o padrão Factory Method está relacionada com uma potencial proliferação e explosão de novas classes no projeto de software. A arquitetura exige que o cliente delegue a responsabilidade de instanciação; portanto, sempre que a aplicação precisar suportar o processamento de uma nova classe de "Produto" (ConcreteProduct), o desenvolvedor será quase sempre forçado a parametrizar o sistema gerando também uma nova subclasse paralela de "Criador" (ConcreteCreator).



Singleton

INCIDÊNCIA EM PROVA: ALTÍSSIMA

Descrição Resumida: esse padrão garante que uma classe tenha apenas uma instância e provê um ponto de acesso global a ela.

O padrão de projeto Singleton é utilizado quando uma aplicação precisa garantir que exista apenas uma instância de determinada classe durante seu ciclo de execução. Em que situações isso faz sentido? Geralmente quando um recurso deve ser compartilhado por toda a aplicação, como um gerenciador de configurações, um serviço de registro de logs ou outro componente cuja duplicação poderia provocar inconsistências ou dificultar o controle de seu estado.

Uma característica importante desse padrão é **a existência de um ponto de acesso global à instância única**. Por que isso é necessário? Porque diferentes partes da aplicação podem precisar utilizar o mesmo objeto, sem criar novas instâncias a cada solicitação. Dessa forma, todos os clientes compartilham exatamente o mesmo objeto, o que mantém o estado centralizado e facilita seu gerenciamento.

É comum encontrar exemplos que associam o Singleton ao gerenciamento de conexões com bancos de dados. Essa associação está sempre correta? Nem sempre, pois aplicações modernas costumam utilizar pools de conexões, nos quais várias conexões permanecem disponíveis para reutilização. Nesses casos, **o Singleton pode ser empregado para gerenciar o pool, mas não necessariamente para representar uma única conexão com o banco de dados**.

O funcionamento do Singleton **baseia-se na restrição da instanciação da classe e na disponibilização de um método responsável por fornecer acesso à única instância existente**. Assim, independentemente da quantidade de solicitações realizadas pela aplicação, todas elas utilizam o mesmo objeto, que pode ser criado no início da execução ou apenas quando for solicitado pela primeira vez, conforme a estratégia adotada na implementação.

```
class Configuracao:
    _instancia = None

    def __new__(cls):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
        return cls._instancia

config1 = Configuracao()
config2 = Configuracao()

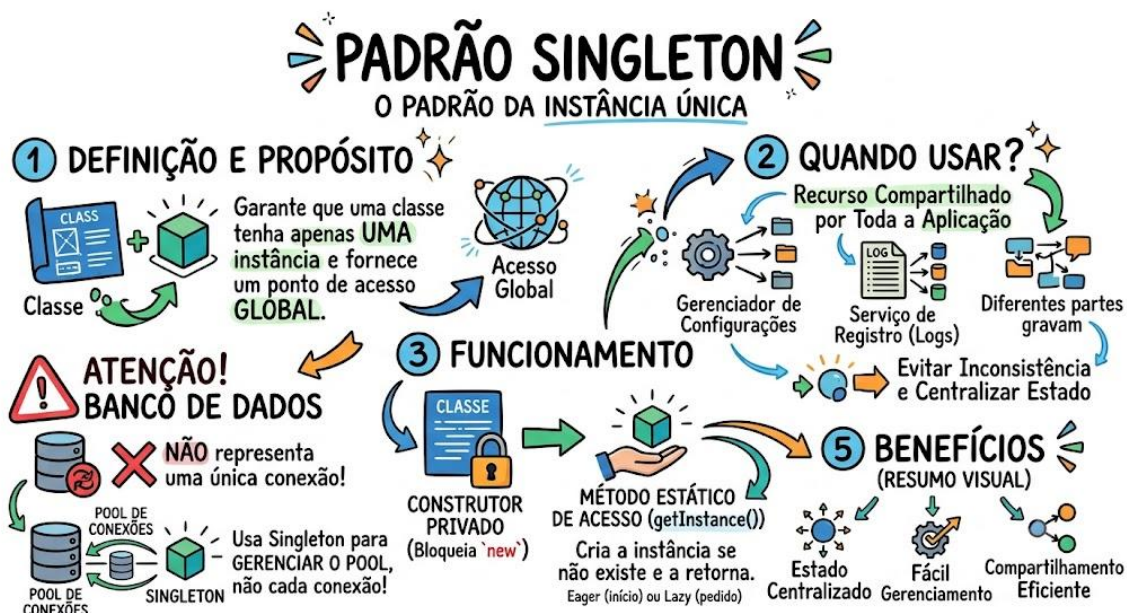
print(config1 is config2)
```

Nesse exemplo, a classe Configuracao implementa o padrão Singleton, garantindo que apenas uma única instância dessa classe seja criada. Isso é feito por meio do método especial __new__(), que verifica se já existe uma instância da classe. Se não existir, ela é criada; caso contrário, a instância já existente é retornada.

A principal ideia do Singleton é assegurar que uma classe possua apenas um único objeto durante toda a execução do programa, fornecendo um ponto único de acesso a ele. Assim, mesmo que o código execute Configuracao() várias vezes, todas as variáveis referenciarão exatamente o mesmo objeto, como demonstrado pela expressão config1 is config2, que retorna True.

Esse padrão é útil quando deve existir apenas uma instância de determinado objeto, como uma configuração global da aplicação, um gerenciador de logs, uma conexão com um banco de dados ou um cache compartilhado. Dessa forma, evita-se a criação desnecessária de múltiplas instâncias e garante-se que toda a aplicação utilize o mesmo objeto.

Apesar das suas utilidades, **o padrão Singleton é amplamente criticado em ambientes de Testes Unitários de software**, sendo comumente **caracterizado pela comunidade moderna como um anti-pattern** (antipadrão) nesse contexto. Como o Singleton introduz um estado global persistente que atravessa todo o ciclo de vida da aplicação, torna-se muito difícil garantir o isolamento e a repetibilidade entre diferentes testes automatizados. A execução de um teste altera os dados estáticos do objeto, o que pode fatalmente "contaminar" e quebrar a execução do próximo teste de maneira imprevisível.



Padrões Estruturais

PADRÕES ESTRUTURAIS

Os Padrões Estruturais são uma categoria dos padrões GoF que se concentram na forma como classes e objetos são organizados para formar estruturas maiores e mais flexíveis. Seu principal objetivo é facilitar a composição de classes e objetos, promovendo baixo acoplamento, reutilização de código e maior facilidade de manutenção. Esses padrões definem maneiras eficientes de estabelecer relacionamentos entre componentes, permitindo que sistemas complexos sejam construídos de forma modular. Fazem parte dessa categoria os padrões Adapter, Bridge, Composite, Decorator, Facade, Flyweight e Proxy.

Adapter

INCIDÊNCIA EM PROVA: MÉDIA

Descrição Resumida: esse padrão converte a interface de uma classe em outra interface que normalmente não poderiam trabalhar juntas por serem incompatíveis.

O padrão de projeto Adapter é utilizado para permitir que classes com interfaces incompatíveis possam trabalhar em conjunto. Como isso acontece? O Adapter atua como um intermediário que recebe chamadas utilizando a interface esperada pelo cliente e as converte para a interface disponibilizada pelo objeto adaptado. Dessa forma, componentes originalmente incompatíveis conseguem cooperar sem que suas implementações precisem ser modificadas.

Uma analogia bastante conhecida é a de um adaptador de tomada. O que fazer quando um equipamento possui um plugue incompatível com a tomada disponível? Em vez de alterar o equipamento ou a instalação elétrica, utiliza-se um adaptador que compatibiliza as interfaces. O mesmo princípio é aplicado em software, pois o Adapter permite integrar componentes desenvolvidos de forma independente.

Considere um sistema responsável por armazenar dados de formulários de visto utilizando o formato de data DD/MM/AAAA, enquanto outro sistema espera receber datas no formato MM/DD/AAAA. Como esses sistemas podem se comunicar sem alterações em suas implementações? Um Adapter pode receber os dados em um formato, realizar a conversão necessária e disponibilizá-los na forma esperada pelo sistema de destino.

Outra dúvida frequente é se o Adapter serve apenas para converter formatos de dados. Essa é apenas uma de suas aplicações. **O objetivo principal do padrão é adaptar interfaces incompatíveis**, permitindo que o código cliente continue utilizando a interface que conhece, enquanto o Adapter traduz as chamadas para a interface do componente que realmente executará a operação.

```
class TomadaEuropeia:
    def fornecer_energia(self):
        print("Fornecendo energia pela tomada europeia.")

class Adaptador:
    def __init__(self, tomada):
        self.tomada = tomada

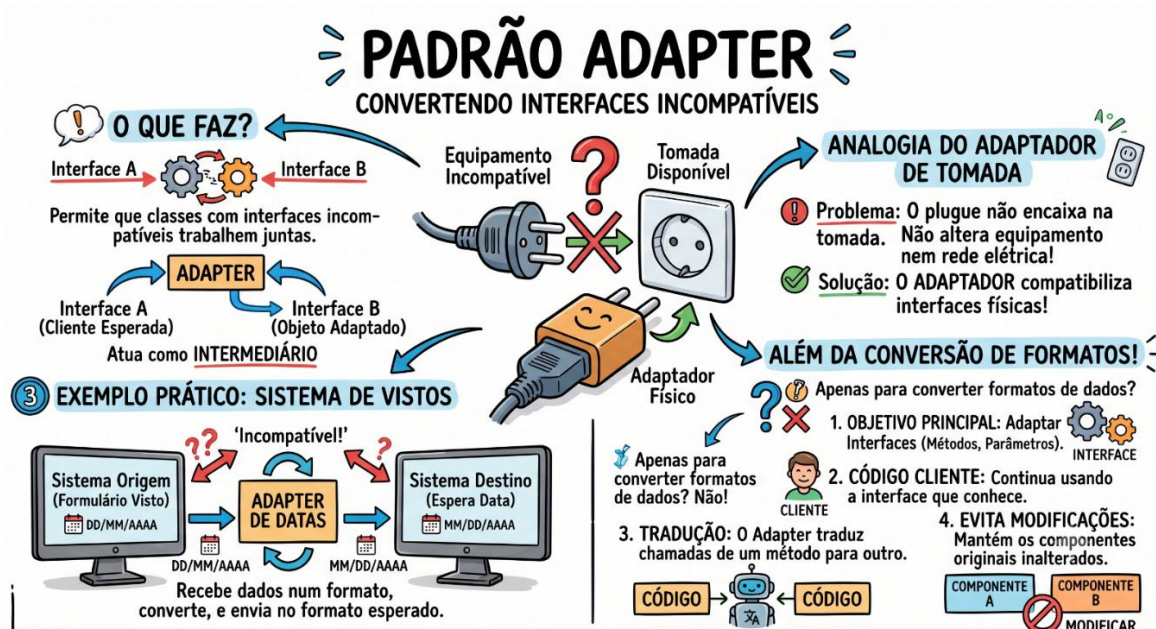
    def ligar(self):
        self.tomada.fornecer_energia()

tomada = TomadaEuropeia()
adaptador = Adaptador(tomada)
adaptador.ligar()
```

Nesse exemplo, a classe TomadaEuropeia representa um componente cuja interface não é compatível com a esperada pelo cliente. A classe Adaptador atua como um intermediário, oferecendo o método ligar(), que é o método esperado pelo cliente, mas internamente delegando a chamada ao método fornecer_energia() da tomada europeia.

A principal ideia do Adapter é permitir que classes com interfaces incompatíveis trabalhem juntas. **Em vez de modificar uma classe já existente, cria-se um adaptador que converte a interface disponível para a interface esperada pelo cliente.** Assim, o código cliente continua funcionando sem precisar conhecer os detalhes da adaptação.

Esse padrão é útil quando se deseja reutilizar componentes existentes cuja interface é diferente da esperada pela aplicação. Exemplos comuns incluem adaptadores de tomadas, integração com bibliotecas de terceiros, sistemas legados, APIs externas e drivers de dispositivos. Em todos esses casos, o Adapter atua como uma "ponte", traduzindo uma interface em outra compatível.



Façade

INCIDÊNCIA EM PROVA: MÉDIA

Descrição Resumida: esse padrão oferece uma interface unificada para um conjunto de interfaces em um subsistema, definindo uma interface de alto nível que facilita a utilização do subsistema.

O padrão de projeto Façade é utilizado para fornecer uma interface unificada e simplificada para um subsistema composto por diversos componentes. Qual é o objetivo dessa abordagem? Permitir que o código cliente acesse as funcionalidades mais comuns sem precisar conhecer os detalhes internos de funcionamento nem a forma como os componentes do subsistema interagem entre si.

Considere uma instituição financeira que possua um sistema legado responsável pelas informações de crédito e um novo sistema que precise utilizar esses dados. Seria conveniente que o novo sistema conhecesse todas as classes e regras do sistema legado? Essa dependência aumentaria a complexidade da integração e tornaria a manutenção mais difícil à medida que o subsistema evoluísse.

O Façade resolve esse problema ao disponibilizar um único ponto de acesso para as operações mais utilizadas. Assim, o cliente realiza suas solicitações por meio dessa interface de alto nível, enquanto a Façade coordena internamente as chamadas necessárias aos componentes do subsistema. Dessa forma, o cliente permanece desacoplado da complexidade existente na implementação.

Outra dúvida frequente é se o Façade impede o acesso direto aos componentes internos do subsistema. Isso não ocorre, pois **seu objetivo não é substituir essas interfaces, mas oferecer uma alternativa mais simples para os casos mais comuns.** Quando necessário, outros componentes da aplicação ainda podem acessar diretamente os serviços específicos do subsistema.

```
class TV:
    def ligar(self):
        print("TV ligada.")

class Som:
    def ligar(self):
        print("Som ligado.")

class HomeTheater:
    def __init__(self):
        self.tv = TV()
        self.som = Som()

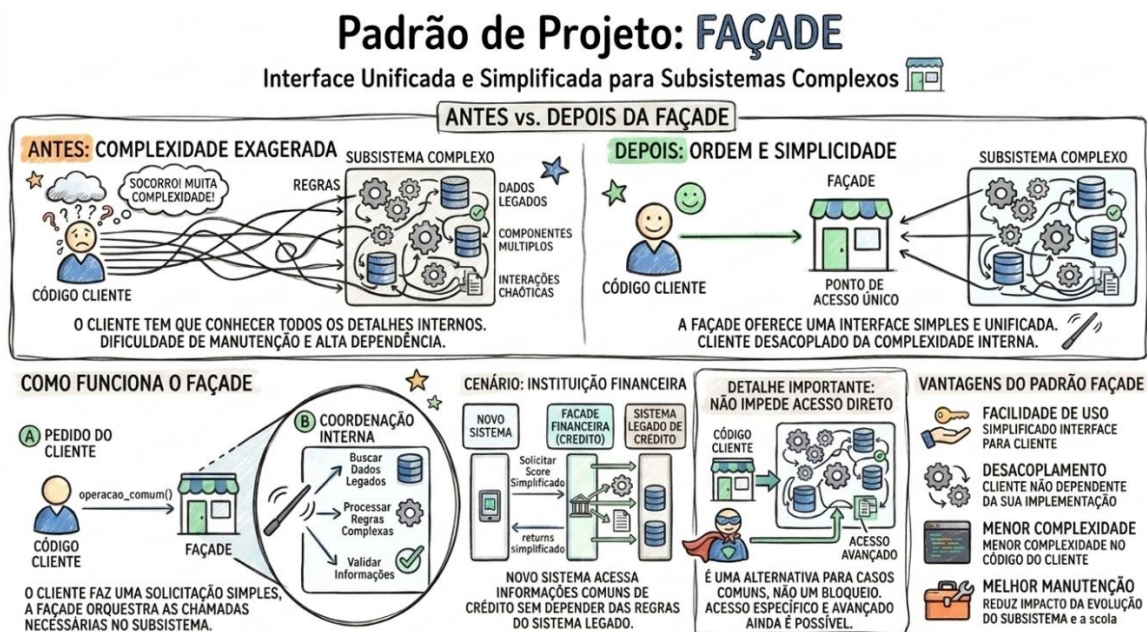
    def assistir_filme(self):
        self.tv.ligar()
        self.som.ligar()

home = HomeTheater()
home.assistir_filme()
```

Nesse exemplo, as classes TV e Som representam componentes do sistema. Para assistir a um filme, seria necessário ligar cada equipamento separadamente. A classe HomeTheater atua como uma Façade, oferecendo um único método (assistir_filme()) que coordena todas as chamadas necessárias aos componentes internos.

A principal ideia do Façade é fornecer uma interface simplificada para um conjunto de classes ou subsistemas mais complexos. Em vez de o cliente precisar conhecer e interagir com vários objetos, ele utiliza apenas a fachada, que centraliza e organiza as chamadas aos componentes internos.

Esse padrão é útil quando um sistema possui muitos objetos ou operações que normalmente são utilizados em conjunto. Exemplos comuns incluem sistemas de home theater, bibliotecas de software, APIs complexas e serviços corporativos, nos quais a fachada oferece um ponto único de acesso, reduzindo a complexidade e o acoplamento do código cliente.



Padrões Comportamentais

PADRÕES COMPORTAMENTAIS

Os Padrões Comportamentais são uma categoria dos padrões GoF que se concentram na comunicação, na colaboração e na distribuição de responsabilidades entre objetos durante a execução do sistema. Seu objetivo é definir formas eficientes de interação entre objetos, reduzindo o acoplamento e tornando o comportamento do software mais flexível e fácil de manter. Fazem parte dessa categoria os padrões Chain of Responsibility, Command, Interpreter, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method e Visitor.

Observer

INCIDÊNCIA EM PROVA: MÉDIA

Descrição Resumida: esse padrão define uma dependência um-para-muitos entre objetos para que, quando um objeto mudar de estado, os seus dependentes sejam notificados e atualizados automaticamente.

O padrão de projeto Observer é utilizado quando vários objetos precisam ser informados automaticamente sobre alterações ocorridas em outro objeto. Como isso funciona? **O objeto que sofre as alterações, denominado Subject ou Observable, mantém uma lista de observadores registrados.** Sempre que seu estado é modificado, todos os observadores são notificados para que possam atualizar suas próprias informações.

Imagine uma tabela de classificação do Campeonato Brasileiro acompanhada por diferentes gráficos, como um gráfico de pizza com a quantidade de vitórias, empates e derrotas e outro que apresenta a evolução da posição de um time ao longo da competição. O que acontece quando os resultados da rodada são atualizados? Todas essas representações precisam refletir imediatamente os novos dados para permanecerem consistentes.

O Observer automatiza esse processo ao **estabelecer uma dependência do tipo um para muitos entre a tabela e os gráficos**. Assim, sempre que a tabela sofre uma alteração, todos os observadores registrados recebem uma notificação e executam sua atualização. Dessa forma, a tabela não precisa conhecer a implementação de cada gráfico, bastando interagir com a interface comum implementada pelos observadores.

Outra dúvida frequente é se a quantidade de observadores precisa ser conhecida antecipadamente. Isso não é necessário, pois **novos observadores podem ser registrados ou removidos durante a execução da aplicação**. Essa característica torna o padrão bastante flexível, já que diferentes componentes podem começar ou deixar de acompanhar as alterações do objeto observado sem modificar sua implementação.

```
class Canal:
    def __init__(self):
        self.inscritos = []

    def inscrever(self, usuario):
        self.inscritos.append(usuario)

    def publicar_video(self):
        for usuario in self.inscritos:
            usuario.notificar()

class Usuario:
    def __init__(self, nome):
        self.nome = nome

    def notificar(self):
        print(f"{self.nome} recebeu uma notificação.")

canal = Canal()

canal.inscrever(Usuario("Ana"))
canal.inscrever(Usuario("Carlos"))
canal.publicar_video()
```

Nesse exemplo, a classe Canal representa o objeto observado (Subject), enquanto a classe Usuario representa os observadores (Observers). Os usuários se inscrevem no canal por meio do método inscrever(). Quando o canal publica um novo vídeo, ele percorre sua lista de inscritos e notifica automaticamente cada um deles.

A principal ideia do Observer é **estabelecer uma relação de dependência do tipo um-para-muitos entre objetos**. Assim, quando o estado de um objeto é alterado, todos os objetos que dependem dele são automaticamente notificados, sem que o objeto observado precise conhecer detalhes sobre cada observador.

Esse padrão é muito utilizado em sistemas de notificações, eventos de interfaces gráficas, atualização de painéis em tempo real, publicação de notícias, redes sociais e aplicações baseadas em eventos. Em todos

esses casos, diversos objetos precisam ser avisados automaticamente quando ocorre uma determinada mudança de estado.



Por fim, algumas dicas para "matar" questões rapidamente...

Falou em...	... provavelmente será:
Criar famílias de objetos relacionados...	Abstract factory
Construção de um objeto complexo com diferentes representações...	Builder
Deixar subclasses decidirem...	Factory Method
Criar uma instância prototípica...	Prototype
Apenas uma instância com um ponto global a ela...	Singleton
Converte uma interface em outra, por serem incompatíveis...	Adapter
Desacoplar interface da implementação...	Bridge
Estruturas de árvore em hierarquia parte-todo...	Composite
Anexa responsabilidades adicionais dinamicamente...	Decorator
Interface unificada de alto nível para simplificar outra complexa...	Façade
Compartilhamento para suportar grandes quantidades de objetos...	Flyweight
Prover substituto para controlar um objeto...	Proxy
Evitar o acoplamento dando oportunidade a outros objetos...	Chain of Responsibility
Encapsula requisição de objetos...	Command
Representação de uma gramática...	Interpreter
Interface única para acessar coleções sequencialmente...	Iterator
Encapsula a forma como objetos interagem...	Mediator
Captura o estado interno de um objeto...	Memento
Quando objeto mudar de estado, notifica os dependentes...	Observer
Altera comportamentos quando modificar o estado interno...	State
Família de algoritmos...	Strategy
Esqueleto de algoritmos...	Template Method
Operação a ser realizada sobre uma estrutura de objetos...	Visitor

QUESTÕES COMENTADAS

1. (FGV / AMAZUL - 2026) Em programação orientada a objetos, um desenvolvedor precisa garantir que apenas uma instância de uma classe de conexão com banco de dados seja criada durante toda a execução da aplicação, evitando múltiplas conexões desnecessárias. O padrão de projeto que resolve esse problema é o

- a) Factory Method.
- b) Observer.
- c) Singleton.
- d) Strategy.
- e) Decorator.

Comentários:

(a) Errado. Factory Method trata da criação de objetos por meio de subclasses, mas não tem como foco garantir uma única instância durante toda a execução.

(b) Errado. Observer é usado para notificar objetos sobre mudanças de estado, não para controlar quantidade de instâncias.

(c) Correto. Singleton é o padrão pensado justamente para permitir que exista apenas uma instância de uma classe, com acesso global controlado.

(d) Errado. Strategy serve para encapsular algoritmos e trocar comportamentos em tempo de execução, não para restringir instâncias.

(e) Errado. Decorator adiciona responsabilidades a objetos de forma dinâmica, sem relação com limitar a criação a uma única instância.

Gabarito: Letra C

2. (FGV / AMAZUL - 2026) Uma equipe de desenvolvimento identificou que o código de um módulo de autenticação está sendo duplicado em vários pontos do sistema, dificultando a manutenção. A equipe decide refatorar o código aplicando um padrão que centralize a criação de objetos de autenticação, permitindo que subclasses decidam qual tipo específico de autenticador instanciar (OAuth, LDAP, ou local). O padrão de projeto que atende a essa necessidade é chamado

- a) Abstract Factory.

- b) Factory Method.
- c) Builder.
- d) Prototype.
- e) Singleton.

Comentários:

(a) Errado. Abstract Factory cria famílias de objetos relacionados. Aqui a ideia central é deixar que subclasses escolham qual autenticador concreto será instanciado.

(b) Correto. Factory Method encaixa direitinho nesse caso: ele centraliza a criação e permite que subclasses decidam entre OAuth, LDAP ou autenticação local.

(c) Errado. Builder é mais usado quando o foco está na construção passo a passo de objetos complexos, não na escolha do tipo concreto por subclasses.

(d) Errado. Prototype trabalha com clonagem de objetos já existentes, o que não é o problema descrito no enunciado.

(e) Errado. Singleton garante uma única instância de uma classe, mas não resolve a variação na criação de autenticadores diferentes.

Gabarito: Letra B

3. (FGV / TCE-RR - 2025) Um bom desenvolvedor de sistemas deve ter conhecimento sobre refatoração, padrões de arquitetura e de projeto, princípios SOLID e outros tópicos importantes sobre programação. Em relação a padrões de projeto, considere o seguinte trecho de código:

```
public interface Heroi {  
    void falar();  
}  
  
public class HomemDeFerro implements Heroi {  
    public void falar() {  
        System.out.println("Eu sou o Homem de Ferro, da Marvel!");  
    }  
}  
  
public class SuperHomem implements Heroi {  
    public void falar() {  
        System.out.println("Eu sou o Super Homem, da DC!");  
    }  
}
```



```
    }  
}  
  
public class MarvelOuDC {  
    public static Heroi getHeroi(String nome) {  
        if (nome.equalsIgnoreCase("HomemDeFerro")) {  
            return new HomemDeFerro();  
        } else if (nome.equalsIgnoreCase("SuperHomem"))  
        {  
            return new SuperHomem();  
        }  
    }  
}
```

Marque a opção que corresponde ao padrão de projeto implementado no código acima.

- a) Singleton
- b) Factory
- c) Observer
- d) Flyweight
- e) Builder

Comentários:

- (a) Errado. Singleton trata de garantir uma única instância de uma classe, o que não é o foco do código mostrado.
- (b) Correto. O método getHeroi centraliza a criação dos objetos e devolve a implementação conforme o nome informado, que é a ideia do padrão Factory.
- (c) Errado. Observer envolve relação de notificação entre objetos observadores e observado, algo que não aparece no trecho.
- (d) Errado. Flyweight busca compartilhar objetos para economizar memória, o que não é o caso desse exemplo.
- (e) Errado. Builder é usado para construir objetos complexos em etapas, diferente da simples escolha e criação feita no código.

Gabarito: Letra B

4. (FGV / TCE-PE - 2025) Leia a descrição a seguir.

Trata-se de um padrão que fornece uma interface para criar famílias de objetos relacionados ou dependentes, sem especificar suas classes concretas, seja porque estas ainda são desconhecidas ou para permitir maior escalabilidade no futuro.

O padrão de projeto descrito é o:

- a) Builder.
- b) Singleton.
- c) Abstract Factory.
- d) Decorator.
- e) Template Method.

Comentários:

(a) Errado. Builder foca a construção passo a passo de objetos mais complexos, não a criação de famílias de objetos relacionados.

(b) Errado. Singleton serve para garantir uma única instância de uma classe, o que é outra proposta.

(c) Correto. Abstract Factory cria famílias de objetos relacionados sem depender das classes concretas, exatamente como diz o enunciado.

(d) Errado. Decorator é usado para adicionar responsabilidades a objetos de forma flexível, não para criar famílias de objetos.

(e) Errado. Template Method define o esqueleto de um algoritmo, deixando etapas para subclasses, o que não é o caso aqui.

Gabarito: Letra C

5. (FGV / AgSUS - 2025) Os princípios GRASP (General Responsibility Assignment Software Patterns) são padrões de design de software orientados a objetos que servem como diretrizes para a atribuição de responsabilidades a classes e objetos. Dentre os princípios GRASP podemos identificar os seguintes:

- a) Baixo Acoplamento (Low Coupling, Proteção contra Variações (Protected Variations) e Responsabilidade única (Single Responsibility).
- b) Alta Coesão (High Cohesion), Indireção (Indirection) e Substituição de Liskov (Liskov Substitution).

- c) Controlador (Controller), Especialista na Informação (Information Expert) e Inversão da dependência (Dependency Inversion).
- d) Controlador (Controller), Criador (Creator) e Fabricação Pura/Pura Fabricação (Pure Fabrication).
- e) Baixa Coesão (Low Cohesion), Indireção (Indirection) e Polimorfismo (Polymorphism).

Comentários:

- (a) Errado. A alternativa mistura princípios GRASP com outro princípio de design, como Responsabilidade Única, que é mais associado ao SOLID.
- (b) Errado. Embora traga itens de GRASP, inclui Substituição de Liskov, que pertence ao SOLID, então o conjunto não fica correto.
- (c) Errado. Controlador e Especialista na Informação são GRASP, mas Inversão de Dependência é princípio do SOLID, não entra aqui.
- (d) Correto. Controlador, Criador e Fabricação Pura são, sim, princípios clássicos do GRASP, cobrados justamente para orientar a distribuição de responsabilidades.
- (e) Errado. Indireção e Polimorfismo aparecem no contexto de GRASP, mas “Baixa Coesão” contraria a ideia do padrão, que trabalha com Alta Coesão.

Gabarito: Letra D

6. (FGV / Prefeitura de BH - 2024) Padrões de projeto são soluções consagradas que se baseiam nas estruturas da orientação a objetos para solucionar problemas comuns em projetos de software. Os padrões são agrupados em tipos. Assinale a opção que indica apenas padrões do tipo comportamental.

- a) Decorator, bridge e adapter.
- b) Singleton, prototype e builder.
- c) Iterator, memento e observer.
- d) Command, mediator e facade.

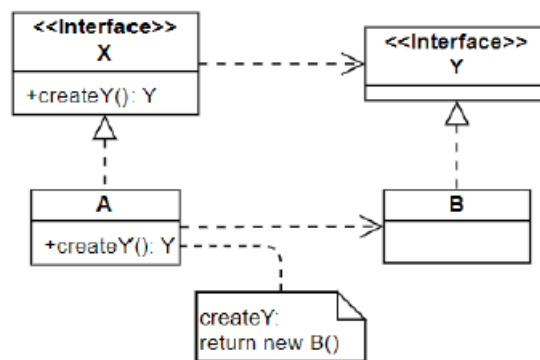
Comentários:

- (a) Errado. Decorator, Bridge e Adapter não entram no grupo dos comportamentais; eles são mais ligados à organização estrutural entre classes e objetos.

- (b) Errado. Singleton, Prototype e Builder são padrões voltados à criação de objetos, ou seja, pertencem ao grupo dos criacionais.
- (c) Correto. Iterator, Memento e Observer são padrões comportamentais, pois tratam da interação entre objetos e da forma como eles se comunicam.
- (d) Errado. Command e Mediator são comportamentais, mas Facade é estrutural, então a alternativa mistura tipos diferentes.

Gabarito: Letra C

7. (FGV / TJ AP - 2024) Padrões de projeto são representados por meio de diagramas da UML, sendo possível identificá-los através da análise de métodos e relacionamentos entre classes ou interfaces. Observe o diagrama de classes apresentado a seguir.



No diagrama é representado o padrão de projeto:

- a) Singleton;
- b) Prototype;
- c) Data Access Object;
- d) Observer;
- e) Abstract Factory.

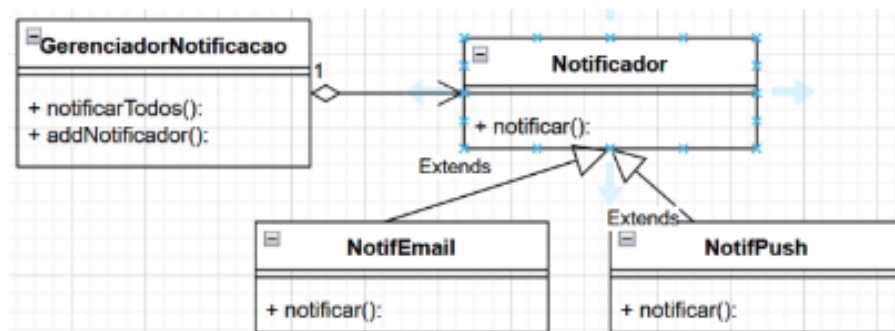
Comentários:

(a) Errado. O padrão Singleton garante a existência de uma única instância de uma classe, normalmente por meio de um construtor privado e um método de acesso à instância. O diagrama não apresenta essas características.

- (b) Errado. O padrão Prototype cria novos objetos por clonagem de uma instância existente, utilizando operações como clone(). O diagrama mostra um método de criação (createY()), e não de clonagem.
- (c) Errado. Data Access Object (DAO) é um padrão de acesso a dados e envolve a abstração das operações de persistência. O diagrama não representa acesso a banco de dados.
- (d) Errado. O padrão Observer modela a relação entre um sujeito e seus observadores, com notificações automáticas de mudanças de estado. Esses elementos não aparecem no diagrama.
- (e) Correto. O diagrama representa o padrão Abstract Factory. A interface X declara o método de fábrica createY(): Y, e a classe A implementa esse método retornando uma instância de B (return new B()), enquanto B implementa a interface Y. Assim, uma fábrica concreta cria um produto concreto por meio de interfaces abstratas, desacoplando a criação dos objetos de sua utilização.

Gabarito: Letra E

8. (FGV / ALETO - 2024) O diagrama a seguir mostra um módulo de notificação para uma aplicação de e-commerce para permitir diversos serviços de notificação, tais como, e-mail e push.



Assinale a opção que indica o padrão de projeto usado nesse módulo.

- a) Singleton.
- b) Factory Method.
- c) Abstract Factory.
- d) Observer.
- e) Prototype.

Comentários:

- (a) Errado. O padrão Singleton garante uma única instância de uma classe. O diagrama não apresenta mecanismo de instância única nem método de acesso global.

- (b) Errado. Factory Method é voltado à criação de objetos por meio de métodos de fábrica. O diagrama não evidencia classes criadoras nem métodos de criação de notificadoros.
- (c) Errado. Abstract Factory cria famílias de objetos relacionados por meio de interfaces de fábrica. O diagrama não mostra fábricas abstratas nem produtos abstratos.
- (d) Correto. O padrão Observer é caracterizado por um objeto manter uma coleção de observadores e notificá-los sobre determinado evento. No diagrama, o GerenciadorNotificacao agrega notificadoros (addNotificador()) e dispara notificarTodos(), enquanto NotifEmail e NotifPush implementam o comportamento notificar(), permitindo adicionar novos tipos de notificação sem alterar o gerenciador.
- (e) Errado. O padrão Prototype baseia-se na clonagem de objetos existentes, característica que não aparece no diagrama.

Gabarito: Letra D

9. (FGV / ALETO - 2024) Assinale a opção que indica o padrão responsável pela criação de objetos de várias classes graças à especialização de sua classe principal.

- a) Abstract Factory.
- b) Decorator.
- c) Strategy.
- d) Builder.
- e) Adapter.

Comentários:

- (a) Correto. Abstract Factory trata justamente da criação de famílias de objetos relacionados, permitindo gerar objetos de várias classes por meio da especialização da fábrica principal.
- (b) Errado. Decorator serve para adicionar responsabilidades a objetos, sem focar na criação de várias classes por especialização.
- (c) Errado. Strategy organiza algoritmos intercambiáveis, ou seja, o foco é comportamento, não criação de objetos.
- (d) Errado. Builder é voltado à construção passo a passo de objetos complexos, com outra proposta.
- (e) Errado. Adapter é usado para compatibilizar interfaces diferentes, não para criar objetos de várias classes por especialização.

Gabarito: Letra A

10.(FGV / TJ MS - 2024) O analista Lucas está desenvolvendo o sistema orientado a objeto JustiSul. Em determinado módulo do JustiSul, o sistema instancia um objeto da classe Terceirizado de forma dinâmica, em tempo de execução, por meio da interface I3rd. A classe Terceirizado pertence a uma biblioteca proprietária externa, de forma que o JustiSul conhece apenas a especificação de I3rd, e não a de Terceirizado. Dentro de outro módulo, o sistema precisa instanciar um segundo objeto Terceirizado. Devido ao alto custo de uma nova inicialização dinâmica, Lucas optou por chamar o método cp3rd do objeto Terceirizado já criado. O cp3rd, declarado em I3rd e implementado na classe Terceirizado, retorna uma nova instância da classe, com um estado idêntico ao do objeto chamador.

Ao permitir a criação de novas instâncias pelo método cp3rd de uma instância existente, Terceirizado implementa o padrão de projeto:

- a) builder;
- b) singleton;
- c) prototype;
- d) factory method;
- e) abstract factory.

Comentários:

- (a) Errado. Builder é usado para montar objetos complexos passo a passo, e não para gerar cópias de uma instância já existente.
- (b) Errado. Singleton busca garantir uma única instância da classe, o que não combina com a criação de um novo objeto por cópia.
- (c) Correto. Prototype é justamente o padrão em que um objeto cria outro a partir de si mesmo, clonando seu estado por meio de uma operação como cp3rd.
- (d) Errado. Factory Method trata da criação por subclasses ou métodos-fábrica, não da duplicação de um objeto já instanciado.
- (e) Errado. Abstract Factory cria famílias de objetos relacionados, mas não tem como foco copiar uma instância existente.

Gabarito: Letra C

11.(FGV / Prefeitura de Caraguatatuba - 2024) Leia o fragmento a seguir sobre Design Patterns.

Os padrões de design desempenham um papel crucial na arquitetura de software, proporcionando soluções testadas e comprovadas para desafios recorrentes no desenvolvimento de sistemas. Esses padrões são geralmente classificados em três categorias principais: padrões de design _____, que são estratégias que se concentram nos padrões de comunicação e colaboração entre objetos; padrões de design _____, que oferece abordagens flexíveis e reutilizáveis na instanciação e composição de objetos e os padrões de design _____, que referem as estratégias voltadas para a eficiente organização de classes e objetos, visando aprimorar a composição e simplificar a interação entre os componentes de um sistema. Cada categoria oferece um conjunto valioso de diretrizes e abordagens que promovem a _____, e manutenção do código, contribuindo para o desenvolvimento de sistemas robustos e eficientes.

Assinale a opção cujos itens completam corretamente as lacunas do fragmento lido.

- a) comportamental – criacional – estrutural – flexibilidade e segurança
- b) estrutural – criacional – comportamental – reusabilidade e segurança
- c) estrutural – comportamental – criacional – flexibilidade e reusabilidade
- d) comportamental – criacional – estrutural – flexibilidade e reusabilidade
- e) criacional – comportamental – estrutural – reusabilidade e acessibilidade

Comentários:

(a) Errado. A sequência inicial até coincide em parte, mas o fechamento com “segurança” não acompanha a ideia central do texto, que destaca ganhos mais ligados ao reaproveitamento e adaptação do código.

(b) Errado. A ordem das categorias não bate com as definições do enunciado. O trecho fala primeiro de comunicação entre objetos, depois de criação e composição, e por fim de organização estrutural.

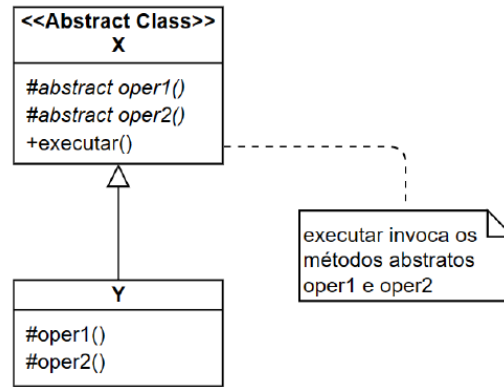
(c) Errado. Aqui há troca entre as categorias do meio e do fim. Além disso, a combinação apresentada não acompanha a sequência conceitual descrita no fragmento.

(d) Correto. “Comportamental – criacional – estrutural – flexibilidade e reusabilidade” encaixa direitinho nas definições dadas e nos benefícios clássicos associados aos design patterns.

(e) Errado. A ordem das categorias não corresponde ao texto, e “acessibilidade” não é o benefício mais diretamente associado ao uso de padrões nesse contexto.

Gabarito: Letra D

12.(FGV / CVM - 2024) Os padrões de projeto são extremamente úteis para organizar a arquitetura do sistema e o modelo de programação. Eles são projetados em diagramas da UML, como no modelo a seguir.



O diagrama expressa o padrão de projeto:

- a) Observer;
- b) Singleton;
- c) Data Access Object;
- d) Intercept Filter;
- e) Template Method.

Comentários:

(a) Errado. O padrão Observer estabelece uma relação entre um objeto e seus observadores para notificações automáticas de mudanças de estado. O diagrama não apresenta esse mecanismo.

(b) Errado. O padrão Singleton garante que uma classe possua apenas uma instância e um ponto global de acesso. Esses elementos não aparecem no diagrama.

(c) Errado. Data Access Object (DAO) é um padrão para abstração do acesso a dados e persistência. O diagrama não representa operações de banco de dados.

(d) Errado. O padrão Intercept Filter organiza filtros para pré e pós-processamento de requisições, característica ausente no diagrama.

(e) Correto. O diagrama representa o padrão Template Method. A classe abstrata X define o método concreto executar(), que estabelece o fluxo do algoritmo chamando os métodos abstratos oper1() e oper2(). A classe Y implementa esses métodos, personalizando partes do algoritmo sem alterar sua estrutura geral.

Gabarito: Letra E

13.(FGV / CVM - 2024) Natália está desenvolvendo um sistema financeiro, no qual os documentos podem receber diferentes tipos de tratamento, e pretende tirar proveito dos padrões de desenvolvimento. Para tratar os documentos, ela criou diversos processos, que serão colocados em uma fila. Assim, o documento passa por um dado processo da fila, é tratado e enviado para o processo seguinte. Adotando essa estratégia, Natália usou o padrão de desenvolvimento denominado:

- a) Chain of Responsibility;
- b) Prototype;
- c) Decorator;
- d) Abstract Factory;
- e) Factory Method.

Comentários:

- (a) Correto. No Chain of Responsibility, a requisição passa por uma sequência de objetos/processos, e cada um trata sua parte antes de encaminhar ao próximo, exatamente como ocorre com os documentos na fila.
- (b) Errado. Prototype é usado para criar novos objetos a partir da clonagem de um modelo existente, não para encadear etapas de processamento.
- (c) Errado. Decorator serve para adicionar responsabilidades a um objeto de forma dinâmica, sem envolver esse fluxo em fila entre vários processos.
- (d) Errado. Abstract Factory é voltado à criação de famílias de objetos relacionados, não ao tratamento sequencial de documentos.
- (e) Errado. Factory Method também é um padrão de criação, focado em definir a forma de instanciar objetos, e não em organizar processamento em cadeia.

Gabarito: Letra A

14.(FGV / SEF MG - 2023) Os padrões de projeto de software (design patterns) tiram proveito máximo dos pilares da orientação a objetos. Usemos como exemplo o padrão Abstract Factory, que é um padrão de projeto criacional que permite produzir famílias de objetos relacionados sem especificar suas classes concretas. O principal pilar da orientação a objetos usado nesse design pattern é

- a) o polimorfismo.
- b) a herança.
- c) o encapsulamento.
- d) a abstração.
- e) a sublevação.

Comentários:

- (a) Errado. Polimorfismo ajuda no uso de interfaces comuns, mas não é o ponto central destacado no Abstract Factory.
- (b) Errado. Herança pode aparecer na implementação, porém não é o pilar principal explorado pelo padrão.
- (c) Errado. Encapsulamento organiza a ocultação de detalhes, mas não é a base mais marcante desse caso.
- (d) Correto. A abstração é o foco do Abstract Factory, pois o padrão trabalha com interfaces e famílias de objetos sem depender das classes concretas.
- (e) Errado. Sublevação não é um pilar clássico da orientação a objetos.

Gabarito: Letra D

15.(FGV / DPE RS - 2023) Os padrões de projeto fornecem soluções para problemas recorrentes no desenvolvimento de sistemas. Maria está desenvolvendo o sistema ComprasWeb e precisa resolver um problema comum de sistemas Web que é a recepção assíncrona de dados, onde deve ocorrer a atualização dos dados na interface do usuário quando ocorre a resposta do servidor, sem que haja o bloqueio das demais funções da interface.

Para tratar o problema do ComprasWeb, Maria deve usar o padrão de projeto:

- a) Observer;
- b) Chain of Responsibility;
- c) Flyweight;
- d) Data Access Object;
- e) Builder.

Comentários:

- (a) Correto. Observer é o padrão indicado quando uma mudança de estado ou a chegada de uma resposta precisa disparar a atualização automática da interface, sem travar o restante da aplicação.
- (b) Errado. Chain of Responsibility serve para encadear tratadores de requisições, não para atualizar interface de forma assíncrona.
- (c) Errado. Flyweight é usado para economizar memória com compartilhamento de objetos, o que não resolve esse cenário.
- (d) Errado. Data Access Object organiza o acesso a dados e ao banco, mas não trata a atualização assíncrona da interface.
- (e) Errado. Builder é voltado à construção passo a passo de objetos complexos, sem relação direta com resposta assíncrona na interface.

Gabarito: Letra A

LISTA DE QUESTÕES

1. (FGV / AMAZUL - 2026) Em programação orientada a objetos, um desenvolvedor precisa garantir que apenas uma instância de uma classe de conexão com banco de dados seja criada durante toda a execução da aplicação, evitando múltiplas conexões desnecessárias. O padrão de projeto que resolve esse problema é o

- a) Factory Method.
- b) Observer.
- c) Singleton.
- d) Strategy.
- e) Decorator.

2. (FGV / AMAZUL - 2026) Uma equipe de desenvolvimento identificou que o código de um módulo de autenticação está sendo duplicado em vários pontos do sistema, dificultando a manutenção. A equipe decide refatorar o código aplicando um padrão que centralize a criação de objetos de autenticação, permitindo que subclasses decidam qual tipo específico de autenticador instanciar (OAuth, LDAP, ou local). O padrão de projeto que atende a essa necessidade é chamado

- a) Abstract Factory.
- b) Factory Method.
- c) Builder.
- d) Prototype.
- e) Singleton.

3. (FGV / TCE-RR - 2025) Um bom desenvolvedor de sistemas deve ter conhecimento sobre refatoração, padrões de arquitetura e de projeto, princípios SOLID e outros tópicos importantes sobre programação. Em relação a padrões de projeto, considere o seguinte trecho de código:

```
public interface Heroi {  
    void falar();  
}
```

```
public class HomemDeFerro implements Heroi {  
    public void falar() {  
        System.out.println("Eu sou o Homem de Ferro, da Marvel!");  
    }  
}
```

```
public class SuperHomem implements Heroi {
```



```
public void falar() {  
    System.out.println("Eu sou o Super Homem, da DC!");  
}  
  
public class MarvelOuDC {  
    public static Heroi getHeroi(String nome) {  
        if (nome.equalsIgnoreCase("HomemDeFerro")) {  
            return new HomemDeFerro();  
        } else if (nome.equalsIgnoreCase("SuperHomem"))  
        {  
            return new SuperHomem();  
        }  
    }  
}
```

Marque a opção que corresponde ao padrão de projeto implementado no código acima.

- a) Singleton
- b) Factory
- c) Observer
- d) Flyweight
- e) Builder

4. (FGV / TCE-PE - 2025) Leia a descrição a seguir.

Trata-se de um padrão que fornece uma interface para criar famílias de objetos relacionados ou dependentes, sem especificar suas classes concretas, seja porque estas ainda são desconhecidas ou para permitir maior escalabilidade no futuro.

O padrão de projeto descrito é o:

- a) Builder.
- b) Singleton.
- c) Abstract Factory.
- d) Decorator.
- e) Template Method.

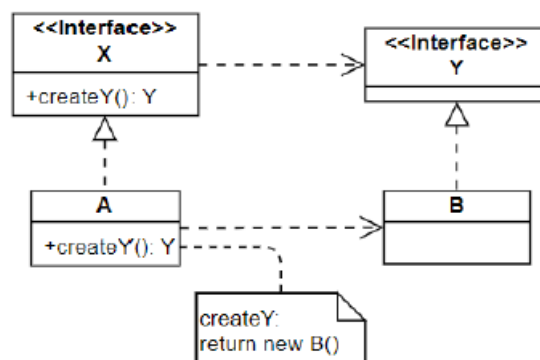
5. (FGV / AgSUS - 2025) Os princípios GRASP (General Responsibility Assignment Software Patterns) são padrões de design de software orientados a objetos que servem como diretrizes para a atribuição de responsabilidades a classes e objetos. Dentre os princípios GRASP podemos identificar os seguintes:

- a) Baixo Acoplamento (Low Coupling), Proteção contra Variações (Protected Variations) e Responsabilidade única (Single Responsibility).
- b) Alta Coesão (High Cohesion), Indireção (Indirection) e Substituição de Liskov (Liskov Substitution).
- c) Controlador (Controller), Especialista na Informação (Information Expert) e Inversão da dependência (Dependency Inversion).
- d) Controlador (Controller), Criador (Creator) e Fabricação Pura/Pura Fabricação (Pure Fabrication).
- e) Baixa Coesão (Low Cohesion), Indireção (Indirection) e Polimorfismo (Polymorphism).

6. (FGV / Prefeitura de BH - 2024) Padrões de projeto são soluções consagradas que se baseiam nas estruturas da orientação a objetos para solucionar problemas comuns em projetos de software. Os padrões são agrupados em tipos. Assinale a opção que indica apenas padrões do tipo comportamental.

- a) Decorator, bridge e adapter.
- b) Singleton, prototype e builder.
- c) Iterator, memento e observer.
- d) Command, mediator e facade.

7. (FGV / TJ AP - 2024) Padrões de projeto são representados por meio de diagramas da UML, sendo possível identificá-los através da análise de métodos e relacionamentos entre classes ou interfaces. Observe o diagrama de classes apresentado a seguir.

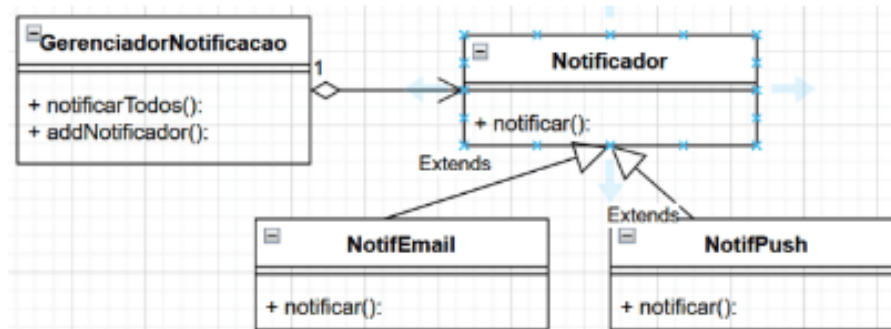


No diagrama é representado o padrão de projeto:

- a) Singleton;
- b) Prototype;
- c) Data Access Object;

- d) Observer;
- e) Abstract Factory.

8. (FGV / ALETO - 2024) O diagrama a seguir mostra um módulo de notificação para uma aplicação de e-commerce para permitir diversos serviços de notificação, tais como, e-mail e push.



Assinale a opção que indica o padrão de projeto usado nesse módulo.

- a) Singleton.
- b) Factory Method.
- c) Abstract Factory.
- d) Observer.
- e) Prototype.

9. (FGV / ALETO - 2024) Assinale a opção que indica o padrão responsável pela criação de objetos de várias classes graças à especialização de sua classe principal.

- a) Abstract Factory.
- b) Decorator.
- c) Strategy.
- d) Builder.
- e) Adapter.

10.(FGV / TJ MS - 2024) O analista Lucas está desenvolvendo o sistema orientado a objeto JustiSul. Em determinado módulo do JustiSul, o sistema instancia um objeto da classe Terceirizado de forma dinâmica, em tempo de execução, por meio da interface I3rd. A classe Terceirizado pertence a uma biblioteca proprietária externa, de forma que o JustiSul conhece apenas a especificação de I3rd, e não a de Terceirizado. Dentro de outro módulo, o sistema precisa instanciar um segundo objeto Terceirizado. Devido ao alto custo de uma nova inicialização dinâmica, Lucas optou por chamar o método `cp3rd` do objeto Terceirizado já criado. O `cp3rd`, declarado em I3rd e implementado na classe Terceirizado, retorna uma nova instância da classe, com um estado idêntico ao do objeto chamador.

Ao permitir a criação de novas instâncias pelo método cp3rd de uma instância existente, Terceirizado implementa o padrão de projeto:

- a) builder;
- b) singleton;
- c) prototype;
- d) factory method;
- e) abstract factory.

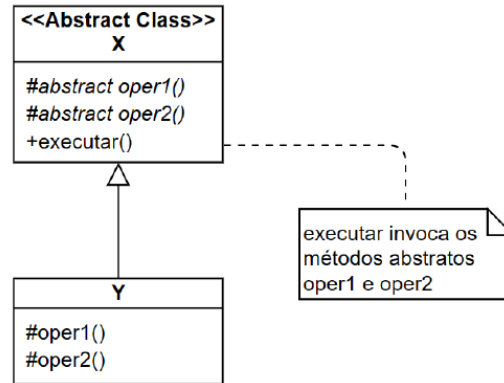
11.(FGV / Prefeitura de Caraguatatuba - 2024) Leia o fragmento a seguir sobre Design Patterns.

Os padrões de design desempenham um papel crucial na arquitetura de software, proporcionando soluções testadas e comprovadas para desafios recorrentes no desenvolvimento de sistemas. Esses padrões são geralmente classificados em três categorias principais: padrões de design _____, que são estratégias que se concentram nos padrões de comunicação e colaboração entre objetos; padrões de design _____, que oferece abordagens flexíveis e reutilizáveis na instanciamento e composição de objetos e os padrões de design _____, que referem as estratégias voltadas para a eficiente organização de classes e objetos, visando aprimorar a composição e simplificar a interação entre os componentes de um sistema. Cada categoria oferece um conjunto valioso de diretrizes e abordagens que promovem a _____, e manutenção do código, contribuindo para o desenvolvimento de sistemas robustos e eficientes.

Assinale a opção cujos itens completam corretamente as lacunas do fragmento lido.

- a) comportamental – criacional – estrutural – flexibilidade e segurança
- b) estrutural – criacional – comportamental – reusabilidade e segurança
- c) estrutural – comportamental – criacional – flexibilidade e reusabilidade
- d) comportamental – criacional – estrutural – flexibilidade e reusabilidade
- e) criacional – comportamental – estrutural – reusabilidade e acessibilidade

12.(FGV / CVM - 2024) Os padrões de projeto são extremamente úteis para organizar a arquitetura do sistema e o modelo de programação. Eles são projetados em diagramas da UML, como no modelo a seguir.



O diagrama expressa o padrão de projeto:

- a) Observer;
- b) Singleton;
- c) Data Access Object;
- d) Intercept Filter;
- e) Template Method.

13.(FGV / CVM - 2024) Natália está desenvolvendo um sistema financeiro, no qual os documentos podem receber diferentes tipos de tratamento, e pretende tirar proveito dos padrões de desenvolvimento. Para tratar os documentos, ela criou diversos processos, que serão colocados em uma fila. Assim, o documento passa por um dado processo da fila, é tratado e enviado para o processo seguinte. Adotando essa estratégia, Natália usou o padrão de desenvolvimento denominado:

- a) Chain of Responsibility;
- b) Prototype;
- c) Decorator;
- d) Abstract Factory;
- e) Factory Method.

14.(FGV / SEF MG - 2023) Os padrões de projeto de software (design patterns) tiram proveito máximo dos pilares da orientação a objetos. Usemos como exemplo o padrão Abstract Factory, que é um padrão de projeto criacional que permite produzir famílias de objetos relacionados sem especificar suas classes concretas. O principal pilar da orientação a objetos usado nesse design pattern é

- a) o polimorfismo.
- b) a herança.
- c) o encapsulamento.
- d) a abstração.

e) a sublevação.

15.(FGV / DPE RS - 2023) Os padrões de projeto fornecem soluções para problemas recorrentes no desenvolvimento de sistemas. Maria está desenvolvendo o sistema ComprasWeb e precisa resolver um problema comum de sistemas Web que é a recepção assíncrona de dados, onde deve ocorrer a atualização dos dados na interface do usuário quando ocorre a resposta do servidor, sem que haja o bloqueio das demais funções da interface.

Para tratar o problema do ComprasWeb, Maria deve usar o padrão de projeto:

- a) Observer;
- b) Chain of Responsibility;
- c) Flyweight;
- d) Data Access Object;
- e) Builder.

GABARITO

- | | | | | | |
|-----------|---------|------------|---------|------------|---------|
| 1. | LETRA C | 6. | LETRA C | 11. | LETRA D |
| 2. | LETRA B | 7. | LETRA E | 12. | LETRA E |
| 3. | LETRA B | 8. | LETRA D | 13. | LETRA A |
| 4. | LETRA C | 9. | LETRA A | 14. | LETRA D |
| 5. | LETRA D | 10. | LETRA C | 15. | LETRA A |

ESSA LEI TODO MUNDO CONHECE: PIRATARIA É CRIME.

Mas é sempre bom revisar o porquê e como você pode ser prejudicado com essa prática.



1 Professor investe seu tempo para elaborar os cursos e o site os coloca à venda.



2 Pirata divulga ilicitamente (grupos de rateio), utilizando-se do anonimato, nomes falsos ou laranjas (geralmente o pirata se anuncia como formador de "grupos solidários" de rateio que não visam lucro).



3 Pirata cria alunos fake praticando falsidade ideológica, comprando cursos do site em nome de pessoas aleatórias (usando nome, CPF, endereço e telefone de terceiros sem autorização).



4 Pirata compra, muitas vezes, clonando cartões de crédito (por vezes o sistema anti-fraude não consegue identificar o golpe a tempo).



5 Pirata fere os Termos de Uso, adultera as aulas e retira a identificação dos arquivos PDF (justamente porque a atividade é ilegal e ele não quer que seus fakes sejam identificados).



6 Pirata revende as aulas protegidas por direitos autorais, praticando concorrência desleal e em flagrante desrespeito à Lei de Direitos Autorais (Lei 9.610/98).



7 Concurseiro(a) desinformado participa de rateio, achando que nada disso está acontecendo e esperando se tornar servidor público para exigir o cumprimento das leis.



8 O professor que elaborou o curso não ganha nada, o site não recebe nada, e a pessoa que praticou todos os ilícitos anteriores (pirata) fica com o lucro.



Deixando de lado esse mar de sujeira, aproveitamos para agradecer a todos que adquirem os cursos honestamente e permitem que o site continue existindo.